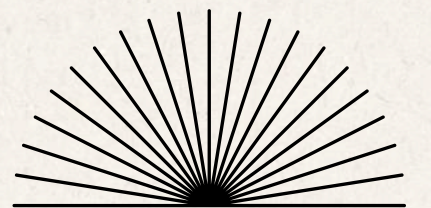




SKIN LESION SEGMENTATION

NAME OF PROJECT:
Skin Lesion Segmentation
using Spectral Generalized
Transformer

PRESENTED BY:
Anushka Garg



Agenda

03	Introduction
04	Problem Statement
05	Approach
06	Architecture Overview
10	Dataset
11	Metrics
12	Results
13	Summary

Introduction

03/11

- Skin cancer is among the most common and life-threatening cancers worldwide
- Early detection is critical for survival
- Dermoscopy images help in diagnosis
- Manual segmentation:
 - Time-consuming
 - Subjective
 - Depends on expertise
- Need: Automated, accurate segmentation

Problem Statement

- Poor generalization across datasets
 - Performance drops due to:
 - Noise
 - Hair artifacts
 - Patient variability
- Difficulty in handling:
 - Irregular shapes
 - Extreme lesion positions (corner / full image)

Approach

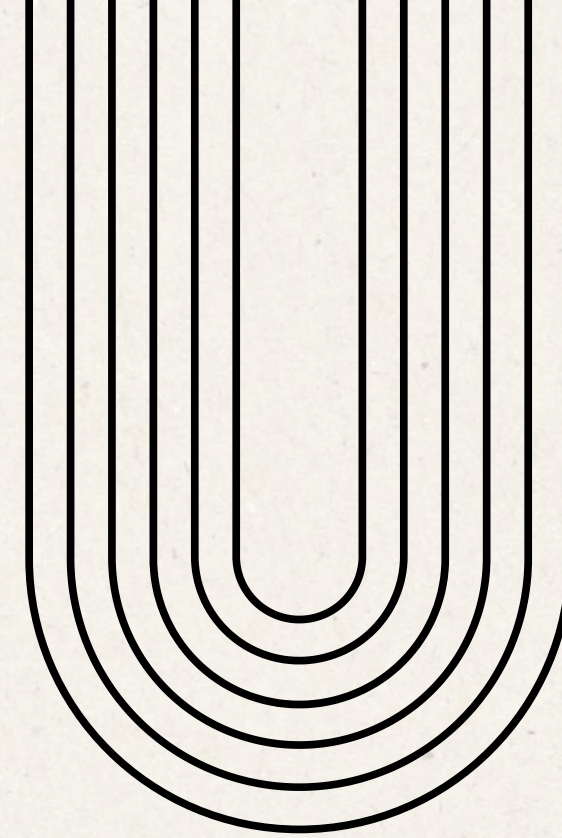
04/11

SGT-Net (Spectral Generalized Transformer)



Combines

- CNN (local features)
- Transformer (global context)
- Frequency-domain learning



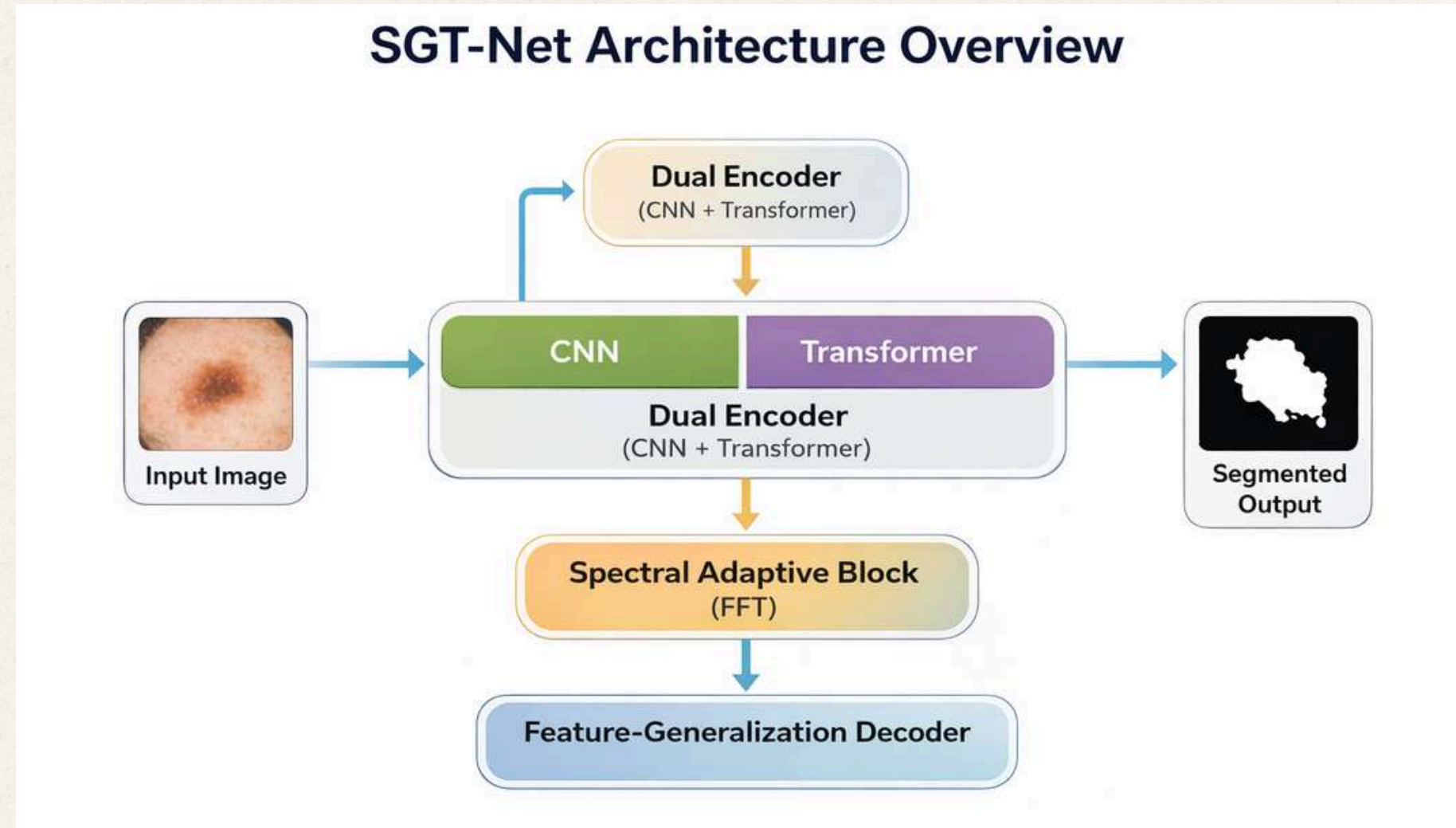
Goal

- Robust segmentation
- Better generalization
- Accurate boundary detection

Architecture Overview

04/11

SGT-Net (Spectral Generalized Transformer)



3 Main Components:

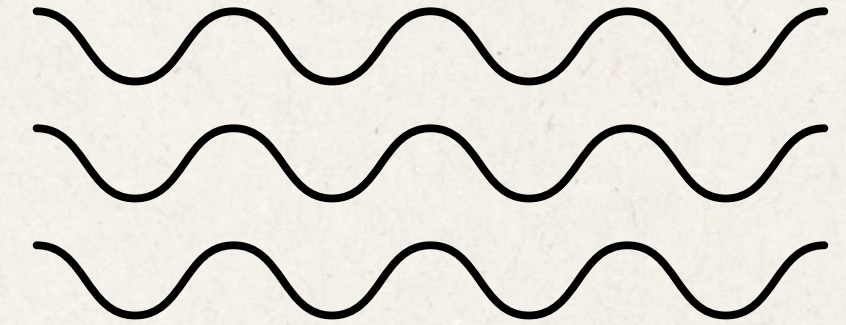
Dual Encoder

Spectral Adaptive Block (SAB)

Feature-generalization Decoder (FD)

Dual Encoder

05/11



- Two parallel branches:
 - ResNet branch
 - Captures local textures & edges
 - Transformer branch
 - Captures global dependencies via self-attention
- Outputs are concatenated
- Result:
 - Rich feature representation

```
class Pra_FATNet(nn.Module):
    def __init__(self, num_classes=1, input_channels=3, deep_supervision=True):
        super(Pra_FATNet, self).__init__()
        channel = 32
        self.resnet = res2net50_v1b_26w_4s(pretrained=True)
        self.rfb2_1 = RFB_modified(512, channel)
        self.rfb3_1 = RFB_modified(1024, channel)
        self.rfb4_1 = RFB_modified(2048, channel)
        self.affinity_attention = AffinityAttention(in_channels=2048)
        self.firstconv = self.resnet.conv1
        self.firstbn = self.resnet.bn1
        self.firstrelu = self.resnet.relu

        transformer = torch.hub.load('facebookresearch/deit:main', 'deit_base_distilled_patch
        _384', pretrained=True)
        self.patch_embed = transformer.patch_embed
        self.transformers = nn.ModuleList([transformer.blocks[i] for i in range(12)])
        self.conv_seq_img = nn.Conv2d(in_channels=3072, out_channels=2048, kernel_size=1, padding=0)

        self.se = squeeze_excitation_block(in_channels=4096)
        self.conv2d = nn.Conv2d(in_channels=4096, out_channels=2048, kernel_size=1, padding=0)
        self.cb = ConvBlock(512, 32)
```


Spectral Adaptive Block (SAB)

- Converts features to frequency domain using FFT
- Applies learnable weights to:
 - Suppress noise
 - Enhance important frequencies
- Converts back using Inverse FFT
- Advantages:
 - Cleaner features
 - Improved generalization across datasets

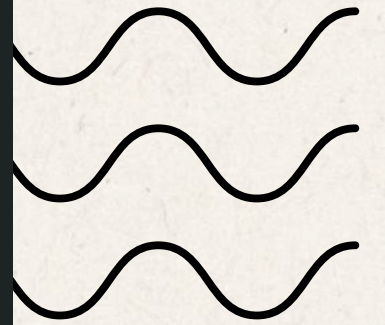
```
class SpectralGatingNetwork(nn.Module):
    def __init__(self, dim, layer_id=None):
        super().__init__()
        if layer_id == 0:
            self.h = 24
            self.w = 13
        elif layer_id == 1:
            self.h = 48
            self.w = 25
        elif layer_id == 2:
            self.h = 96
            self.w = 49
        elif layer_id == 3:
            self.h = 12
            self.w = 7

        self.dim = dim

    def forward(self, x):
        B, C, H, W = x.shape
```

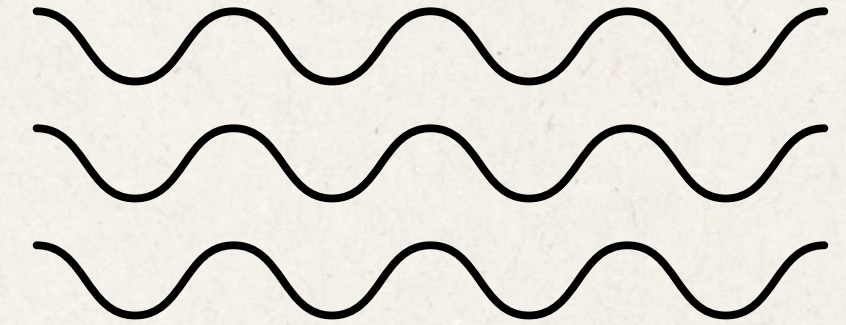
```
class SpatialAttentionBlock(nn.Module):
    def __init__(self, in_channels):
        super(SpatialAttentionBlock, self).__init__()
        self.query = nn.Sequential(
            nn.Conv2d(in_channels, in_channels // 8, kernel_size=(1, 3), padding=(0, 1)),
            nn.BatchNorm2d(in_channels // 8),
            nn.ReLU(inplace=True)
        )
        self.key = nn.Sequential(
            nn.Conv2d(in_channels, in_channels // 8, kernel_size=(3, 1), padding=(1, 0)),
            nn.BatchNorm2d(in_channels // 8),
            nn.ReLU(inplace=True)
        )
        self.value = nn.Conv2d(in_channels, in_channels, kernel_size=1)
        self.gamma = nn.Parameter(torch.zeros(1))
        self.softmax = nn.Softmax(dim=-1)
```

05/11



Feature-generalization Decoder (FD)

05/11



- Uses:
 - Reverse attention Focuses on missed lesion regions
 - Multi-scale fusion Combines features at different resolutions
- Result:
 - Accurate boundaries
 - Handles unusual lesion shapes & positions

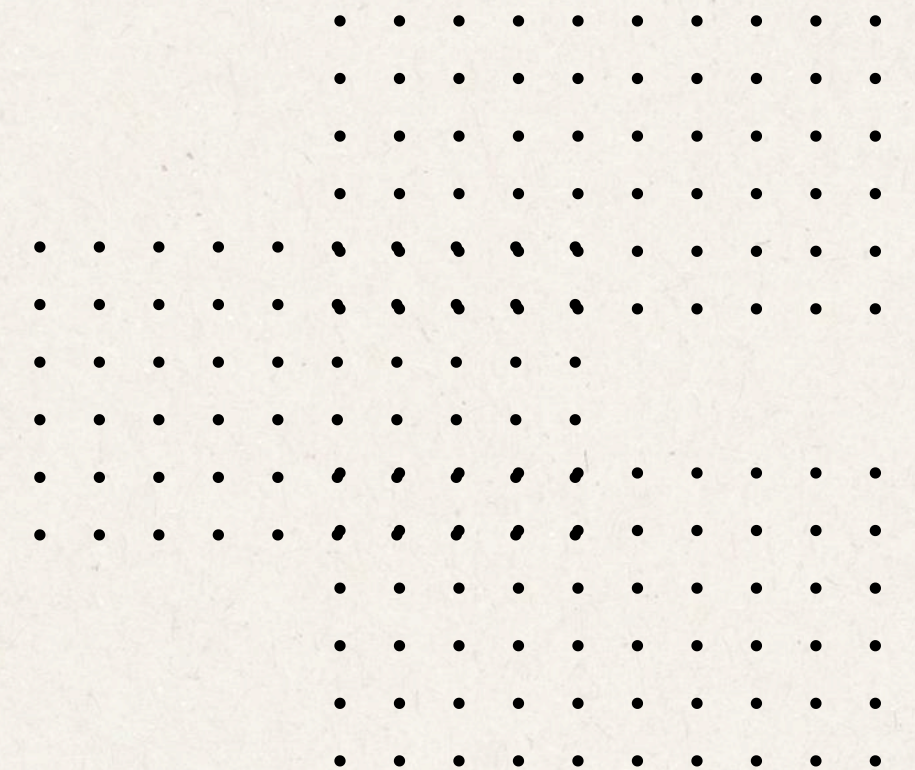
```
class decoderAttention(nn.Module):
    def __init__(self, in_channels, layer_id):
        super(decoderAttention, self).__init__()
        kernel_size = 3
        n_div = 4
        self.dim_conv = in_channels // n_div
        self.dim_untouched = in_channels - self.dim_conv * 3
        self.group = 1
        self.conv1 = nn.Conv2d(self.dim_conv, self.dim_conv, kernel_size,
                                stride=1, padding=(kernel_size - 1) // 2, groups=self.group)
        self.conv2_1 = nn.Conv2d(self.dim_conv, self.dim_conv, (1, 5), stride=1, padding=(0,
        2), groups=self.group)
        self.conv2_2 = nn.Conv2d(self.dim_conv, self.dim_conv, (5, 1), stride=1, padding=(2,
        0), groups=self.group)
        self.conv3_1 = nn.Conv2d(self.dim_conv, self.dim_conv, (1, 7), stride=1, padding=(0,
        3), groups=self.group)
        self.conv3_2 = nn.Conv2d(self.dim_conv, self.dim_conv, (7, 1), stride=1, padding=(3,
        0), groups=self.group)
        self.conv = nn.Conv2d(in_channels, in_channels, kernel_size=1)
        self.sgn = SpectralGatingNetwork(dim=self.dim_untouched, layer_id=layer_id)
```


Dataset

06/11



- **DATASET**
 - **ISIC 2017 DATASET DERMOSCOPIC SKIN IMAGES**
 - **GROUND TRUTH SEGMENTATION MASKS**
- **PREPROCESSING:**
 - **RESIZING**
 - **NORMALIZATION**

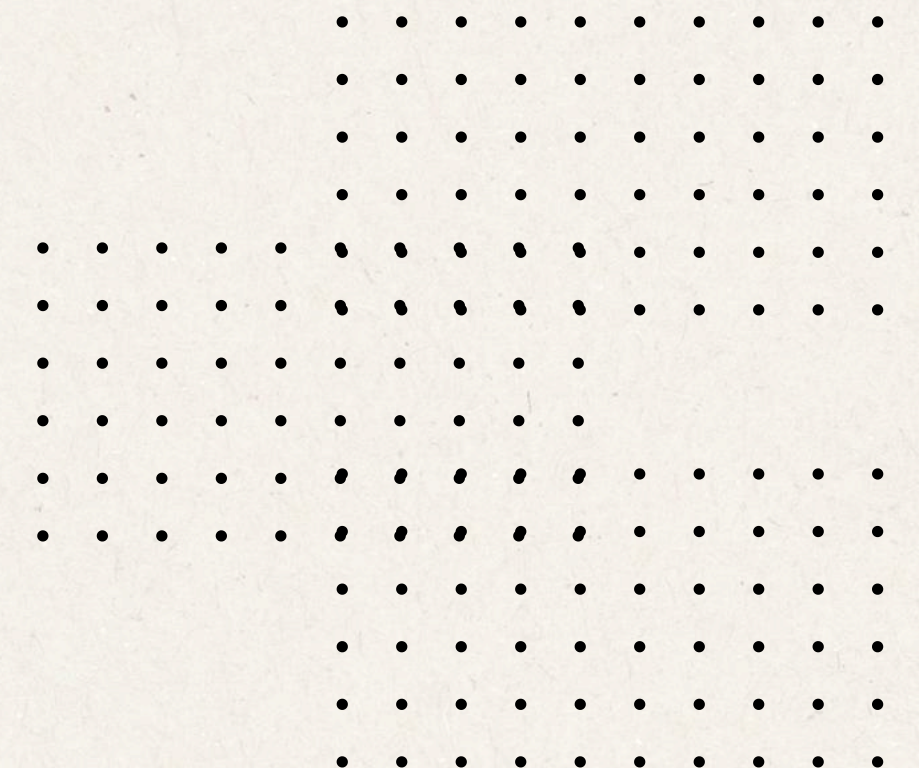


Evaluation Metrics



07/11

- **IoU (Intersection over Union)**
 - **Dice Coefficient**
 - **Loss**
 - **Training Loss**
 - **Validation Loss**
- **Used to measure:**
 - **Accuracy**
 - **Overfitting**
 - **Model convergence**



Results

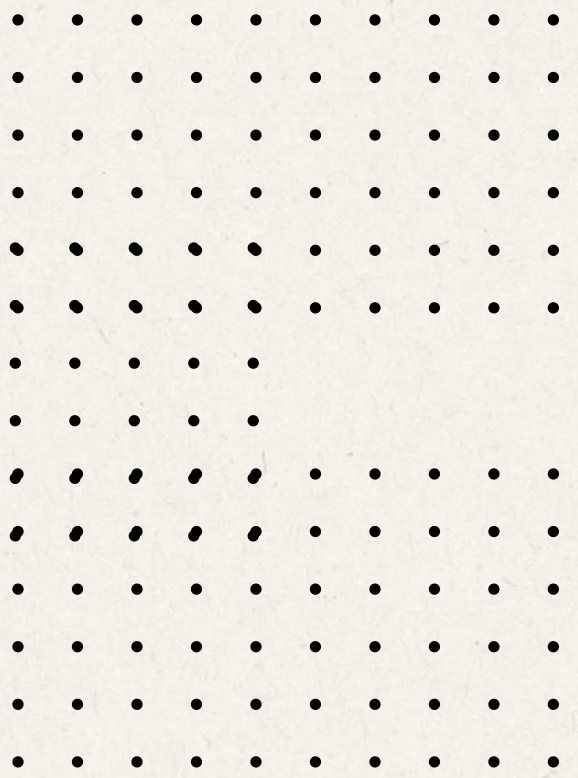
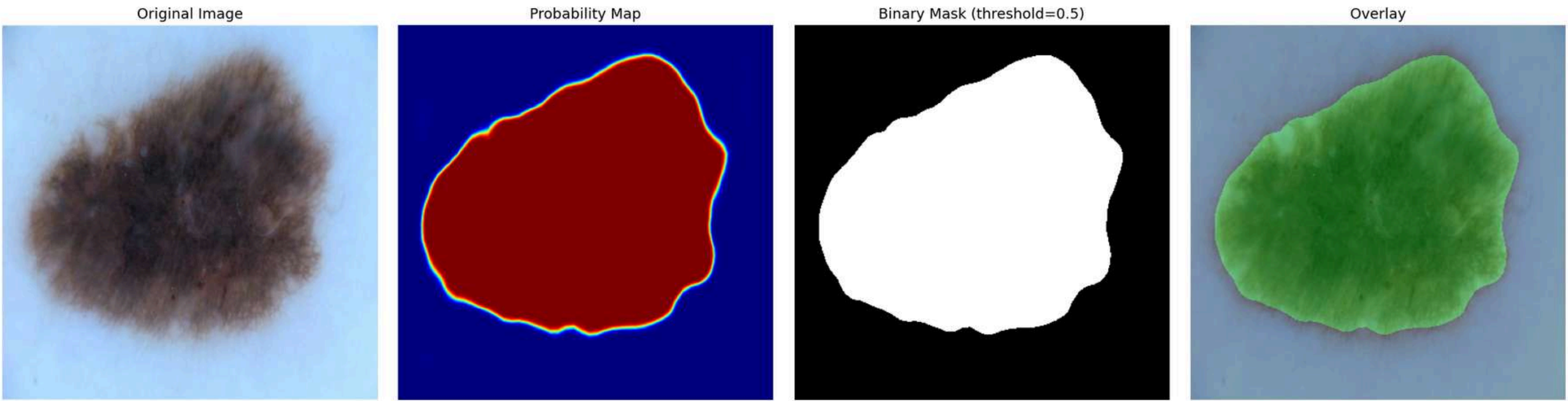
07/11

```
[INFO] Model loaded from /kaggle/input/models/anushka1511/best-model/pytorch/d
x torch.Size([1, 64, 96, 96])
x1 torch.Size([1, 256, 96, 96])
x2 torch.Size([1, 512, 48, 48])
x3 torch.Size([1, 1024, 24, 24])
x4 torch.Size([1, 2048, 12, 12])
emb torch.Size([1, 576, 768])
e3 torch.Size([1, 1024, 24, 24])
e2 torch.Size([1, 512, 48, 48])
e1 torch.Size([1, 256, 96, 96])
feature_out torch.Size([1, 2048, 12, 12])
d4 torch.Size([1, 1024, 24, 24])
d3 torch.Size([1, 512, 48, 48])
d2 torch.Size([1, 256, 96, 96])
ra5_feat torch.Size([1, 1, 48, 48])
ra4_feat torch.Size([1, 1, 12, 12])
ra3_feat torch.Size([1, 1, 24, 24])
ra2_feat torch.Size([1, 1, 48, 48])
d3 torch.Size([1, 1, 48, 48])
```

```
Train Loss: 0.1740
Val   Loss: 0.1883 | Dice: 0.9132 | IoU: 0.8431

✅ Saved best model (Dice=0.9132) → /kaggle/working/chec

[DONE] Best Val Dice: 0.9132
```



Keypoints

- Hybrid CNN + Transformer architecture
- Frequency-domain learning using FFT
- Improved handling of:
 - Noise
 - Irregular lesion shapes
- Strong generalization capability

Summary

- SGT-Net improves skin lesion segmentation by:
 - Combining spatial + spectral learning
 - Enhancing generalization
- Provides reliable support for dermatological diagnosis

Thank You

